AESC v2: Two-Speed Memory Architecture (Revised Technical Report, v3)

Oct 11, 2026 · @Giridhari

Abstract

AESC (Adaptive Event-State Computation) is a recurrent language-model architecture proposed by the author, Giridhari Karmakar. It combines a k-step neural field, a predictive dendrite, top-k skill routing and two-speed memory: fast weights written token by token by a surprise-gated delta rule, and slow weights trained by backpropagation and replay.

The author's hypothesis is that a model that carries a state and a fast-weight matrix across tokens can store and retrieve new associations while it reads. In a 9.92M-parameter reference implementation, the fast memory solved a 4-pair associative-recall task at 1.000 accuracy after 800 steps (chance 0.125). The first 10M implementation scored 0.37 with fast updates on or off, and four of its modules received zero gradient.

This revision describes eleven failure modes found in the first implementation and the changes made for them, audits each component in turn, and adds a safety observability design. Only the fast memory has experimental support so far; the other blocks are unproven.

Author's note

This report describes a research hypothesis and an architecture that I propose. It does not claim that the architecture works in general, and it does not claim that it will work as intended.

What I did. I built a 9.92M-parameter reference implementation and tested it on one small synthetic task, associative recall with 4 key-value pairs. In that run it reached accuracy 1.000. I also audited my first implementation, found eleven failures (Section 9), and revised the design. The component audit in Section 5 and the diagrams were added later. The audit was done from the report and its equations, and I have not yet checked it against the code.

What I do not know. The results are single-seed runs of a small model on a CPU, on one toy task. I do not know whether the slow memory, neuromodulator, metacognition, router or predictor help at all, because that task does not need them. I do not know how the

model behaves on harder tasks, at a larger scale, or against baselines over several seeds.

Where I may be wrong. In my own understanding the design is correct, but that is a belief, not a result. I may be wrong in several places:

- the one-step delay that is meant to prevent label leakage (its test has not been run)
- the predictor target and the risk of embedding collapse
- the router, which may compute every skill and so save no compute
- the slow memory, whose read and write may not use the same key space
- the rule-based controller, which may add nothing
- the safety design, which is not implemented

What I will do. I did my best on this version. If testing shows a problem, I will try to fix it. I will report the problem and the fix in the next revision, and I will keep failed results in the report instead of removing them. A block that shows no benefit on the task built for it will be reported as "no evidence yet", or removed.

1. Hypothesis and proposed architecture

The hypothesis and the architecture in this report are the author's own. Both are proposals; only a small part of them has been tested.

Hypothesis. A small recurrent model with test-time memory can store and retrieve new associations while it reads. It does so because it carries a state and a fast-weight matrix across tokens, where a transformer has no built-in short-term plasticity. The hypothesis is untested beyond one synthetic task.

Claims. The report states two component-level claims that the experiments in Section 8 can test:

1. C1. A fast memory with a differentiable delta-rule write solves associative recall. Supported: accuracy 1.000 on 4-pair recall (Section 8, Run 1).
2. C2. Each added block (slow memory, neuromodulator, metacognition, router, predictor) beats a control with the same skills and without that block, on a task built to need it. Untested.

Proposed architecture. One recurrent state per token. A dendrite reads a local window, a neural field iterates k steps, a router picks skills, and a fast memory is written by a local delta rule while a slow memory is trained by backpropagation and replay. Section 3 draws it and Appendix A gives the equations.

Contributions of this revision.

- A revised design and equations (Section 3, Appendix A)
- A gradient policy for the fast write, backed by a measured ablation (Section 4)
- A component-by-component audit with fixes and first tests (Section 5)
- Results from a reference implementation and an audit of the first one (Section 8)
- A safety observability design (Section 7)

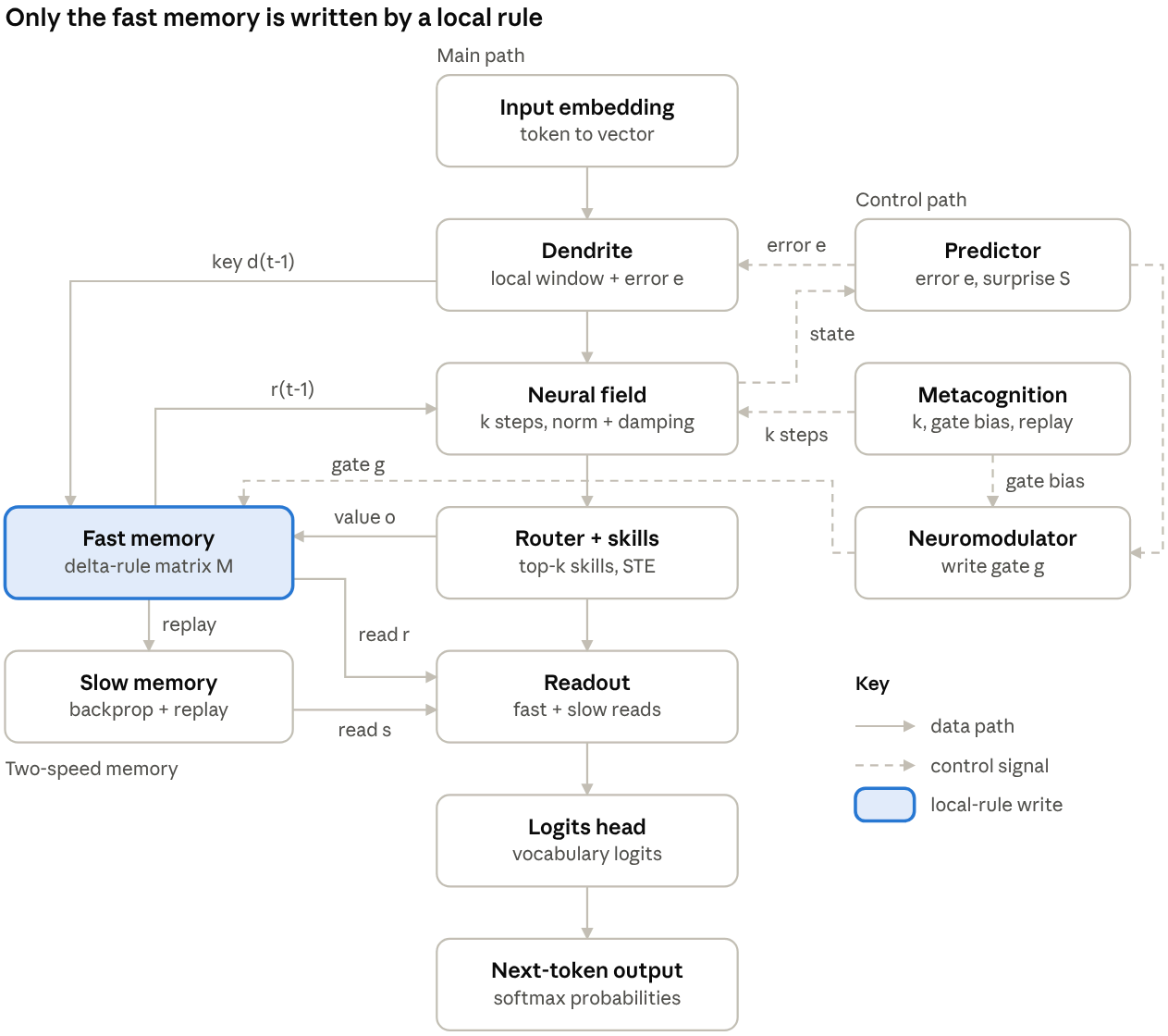
2. Related work

Each component has precedent; the contribution is the combination and the audit signals, not any single part.

- **Fast weights.** Schmidhuber (1992) and Ba et al. (2016) use a fast matrix that changes within a sequence. Schlag et al. (2021) show that linear attention is a fast-weight programmer, and DeltaNet (Yang et al., 2024) uses the delta rule that AESC's write also uses.
- **Test-time memory.** Titans (Behrouz et al., 2024) writes memory driven by surprise, which is close to the AESC gate.
- **State-space models.** Mamba (Gu and Dao, 2023) is the main recurrent baseline for the comparisons in Section 8.
- **Predictive coding.** Rao and Ballard (1999) motivate the dendrite error signal.
- **Two-speed memory.** McClelland, McNaughton and O'Reilly (1995) motivate fast learning plus replay into slow weights.
- **Looped computation.** Universal Transformers (Dehghani et al., 2018) reuse weights over steps, as the k-step field does.
- **Routing.** Sparse mixtures of experts (Shazeer et al., 2017) and straight-through estimators (Bengio et al., 2013) support the router.

3. Proposed architecture

Each token passes through the blocks below in a fixed order, and only the fast memory is written by a local rule. The figure shows the data path, the control path and the two-speed memory.



architecture · main path, control path, two-speed memory

Tokens move down the center column. Dashed arrows are control signals, and surprise S runs from the predictor to the neuromodulator. The highlighted fast memory is the only block written by a local rule.

Per-token loop. The outer loop runs once per token; the inner loop runs the field k_t times. The dendrite output d_t and the memory read r_{t-1} stay fixed inside the inner loop, and the field state h carries over from token to token.

```
h, M, r, e = init                       # once per sequence
for t in tokens:
    x_t = Embed(token_t)
    d_t = Dendrite(x_t, x_{t-1}, ..., e_t)
    S_t, g_t, k_t = Control(e_t, h)
    for _ in range(k_t):                # k-step neural field
        h = Field(h, d_t, r_{t-1})
    o_t = RouterSkills(h)
    M   = Write(M, d_{t-1}, o_t, g_t)  # fast memory, delta rule
    r_t = Read(M, d_t)
    z_t = Readout(o_t, r_t, u_t)       # u_t: slow-memory read
    x_hat_{t+1} = Predictor(h)         # e_{t+1} is formed when x_{t+1}
arrives
```

The slow-memory read u_t goes to the readout only. The field receives only the fast-memory read r_{t-1}.

Block	Role	Learning
Input embedding	Token to vector	Backprop
Dendrite	Local causal window; adds the previous step's prediction error	Backprop (next-token loss + error energy)
Neural field	k-step recurrent state with norm and damping; reads memory from the previous step	Backprop
Predictor, compare, surprise	Predicts the next input; surprise is the mean absolute error, detached	Backprop (error energy)
Neuromodulator	Write gate g from relative surprise and state activity	Gradient through the write (Section 4)
Router and skills	Top-k skills; learned weights kept in the forward pass	Backprop + straight-through
Fast memory	Per-sequence matrix, delta-rule write scaled by g, learned decay	Local rule in form; gradient to key, value and gate inside a truncated window
Slow memory	Consolidated weights read by a query	Backprop + replay

Block	Role	Learning
Metacognition	Chooses k, gate bias and replay timing from surprise statistics	Rules (a learned policy is future work)
Readout and logits head	Adds fast and slow reads to the skill output; vocabulary logits, then softmax gives the next-token output	Backprop

Timing rule (intended to avoid leakage). The predictor forecasts x(t+1) from the state after token t. The error is computed when x(t+1) arrives and is used from step t+1 on. Nothing computed from x(t+1) may reach the logits that predict x(t+1).

Memory addressing. The write key comes from the previous step's dendrite feature, and the write value comes from the current skill output. The read query comes from the current dendrite feature, so a stored pair is retrieved when its key token appears again.

Notation note. Appendix A uses k for three things: field steps k_t, the number of routed skills, and the memory key k_t. Renaming them (K_field, K_route, key_t) is planned.

Working flows

Three flows show how the architecture runs: one token, one training step, and the safety monitor.

One token.

One token: nine steps, with the field repeated k(t) times

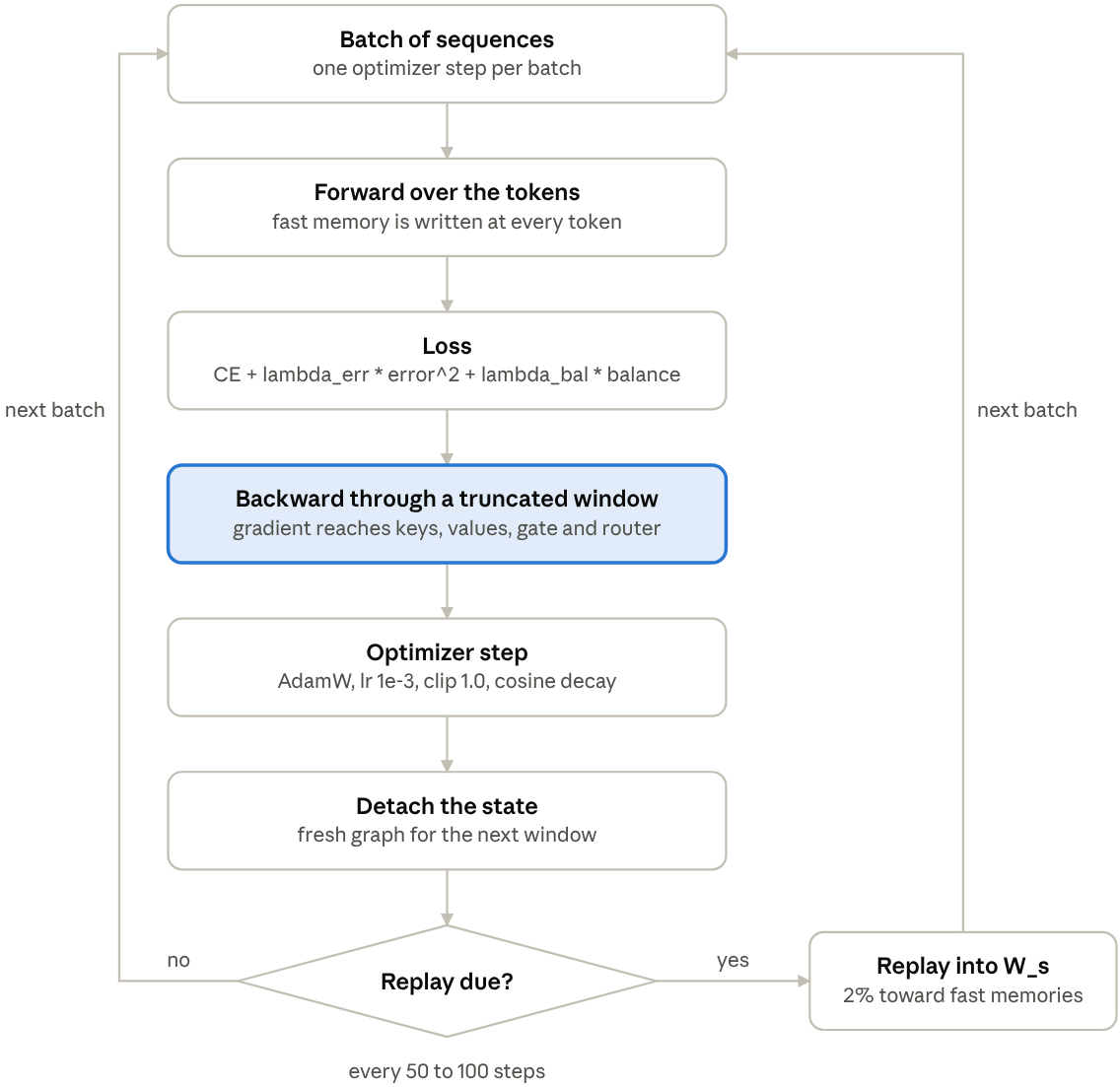


per-token flow · nine steps, one inner loop

Each token runs the nine steps in order. The field step repeats k(t) times, and the predictor output waits for the next token before it becomes an error.

One training step.

One training step: the write stays in the graph, replay runs every 50 to 100 steps

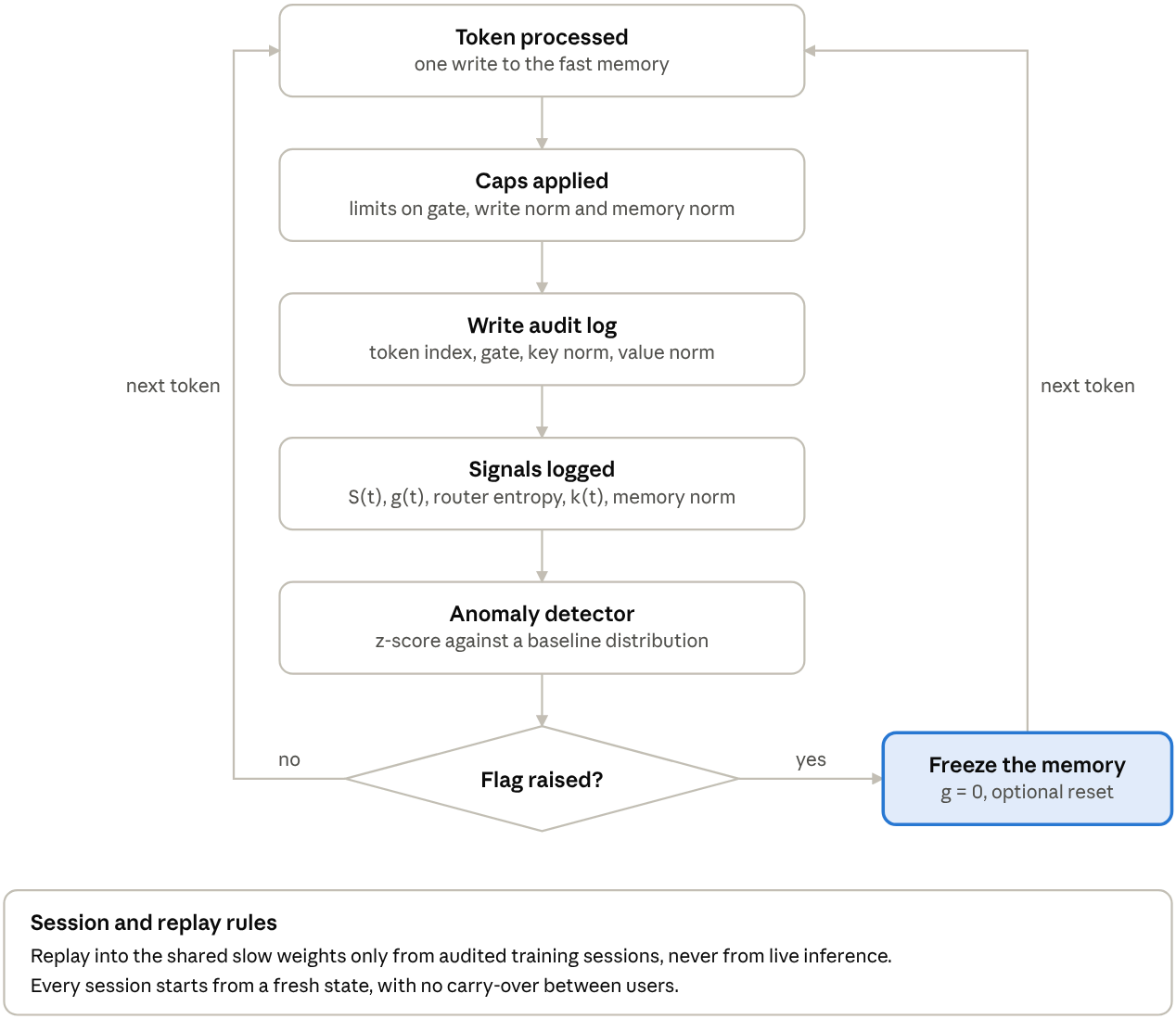


training step flow · one decision

The highlighted step is the gradient policy of Section 4: the delta-rule write stays inside the graph for a truncated window, and the state is detached between windows.

Safety monitor.

A flagged write freezes the memory, and replay never comes from live sessions



safety monitor flow · one decision

The monitor runs on every token. A raised flag sets the gate to zero so the memory only decays. This is a design; it has not been implemented or evaluated (Section 7).

4. Training procedure

Training takes one optimizer step per batch on the next-token loss, and the fast memory is written at every token. The write stays inside the gradient graph for the key map, the value map and the gate.

```
L = \sum_t \mathrm{CE}(z_t, y_t) + \lambda_{err}\,\lVert e_t \rVert^2 +
\lambda_{bal}\, B(\pi_t)
```

B is a router balance term that replaces the entropy term of v1. The reference implementation uses lambda_err = 0.1 and lambda_bal = 0.01.

Gradient policy for the fast write. v1 (Proposition 3) put a stop-gradient on the write, which cuts all gradient to W_k, W_v and the gate. In the reference implementation a stop-gradient write gave recall 0.75 against 1.000 with the write inside the graph, and the neuromodulator gradient was exactly zero. The policy is therefore to keep the delta-rule write differentiable inside a truncated window and to detach the state between windows. Purely local credit assignment stays a research direction; if it is pursued, the neuromodulator needs its own auxiliary loss, for example a gate target based on how much the write lowers the next-token loss. That loss is a proposal and has not been tested.

Routing. The forward pass uses the softmax over the top-k logits, so unequal weights survive. The backward pass uses the gradient of the full softmax (straight-through). In a unit test, hard top-k gave router gradient to only the 2 picked skills out of 4, and straight-through gave it to all 4.

Replay and optimizer. Every 50 to 100 steps, Metacognition triggers replay: the slow weights move 2% toward the surprise-weighted average of the fast memories in the batch. The reference notebook uses AdamW with learning rate 1e-3, 100 warmup steps, cosine decay to 10%, and gradient clipping at 1.0. The 1e-4 to 3e-4 range in v1 was not tested. A curriculum of structured tasks first is still recommended and still untested.

Changes proposed by the audit (Section 5), not yet tested.

- Detach the target in the error: e = sg(x_{t+1}) - x_hat, so the loss cannot shrink the error by shrinking the embeddings.
- Warm up lambda_err from 0 to 0.1 over the first 500 steps, and compare against feeding the predictor a detached state.
- Initialise h_0, M_0 and r_0 to zeros, and give the running surprise average a defined start (S_tilde = 1 at the first step).

5. Component-by-component audit

Only the fast memory has experimental support; every other block is either unproven or has an open design problem. The table comes from the reported runs and from reading the equations in Appendix A. It is a design-level audit: the implementation code was not reviewed, so each finding still has to be confirmed there.

Block	Status	Main problem	Fix to try	First test
Dendrite	Gradient nonzero (Run 6); supplies the memory key and query	The error term gamma*e enters the key, so one token can get different keys in different contexts; the leakage test is a design review, not a run	Run the leakage test; build the key from the window part only	gamma = 0 against gamma > 0; log the norm of d_t
Predictor	Gradient nonzero; no effect on recall	Embedding collapse if the target is not detached; random tokens give no surprise signal; start of the surprise average undefined	Detach the target; warm up lambda_err; define the start value	Surprise on ABCABC against random tokens; embedding norm per epoch
Neural field	Gradient nonzero; reads the memory	The slow-memory read never reaches the field; r_{t-1} arrives one step late; h_0 undefined	Document the initial state; ablate a slow-memory term in the field	Multi-hop lookup with fixed against adaptive k

Block	Status	Main problem	Fix to try	First test
Neuromodulator	Gate receives gradient because the write is in the graph	Few learnable parameters (a_s, a_c, b); the bias delta reuses the same surprise signal as the gate; the gate can saturate; one scalar per token	Initialise g near 0.5; clamp g; optional content-dependent gate	Histogram of g; noisy stream with few important tokens; learned g against fixed g
Router and skills	Straight-through gives gradient to all 4 skills	Top-k is dense compute, not sparse; the gradient is biased; collapse onto one skill; balance term on pi, not on the selection	Choose dense or sparse and report FLOPs; balance on selection fraction	Skill usage histogram; task needing copy, reverse and count
Fast memory	Recall 1.000 on 4 pairs; about 0.40 without it	Decay and delta rule are inconsistent; capacity is about d_k pairs; the read follows the write, so the new value can be read back; decay range unconstrained	Delta rule on Lambda*M; Lambda = sigmoid(theta); cap g and the memory norm	8 to 64 pairs with distractors; overwrite task; read-before-write variant

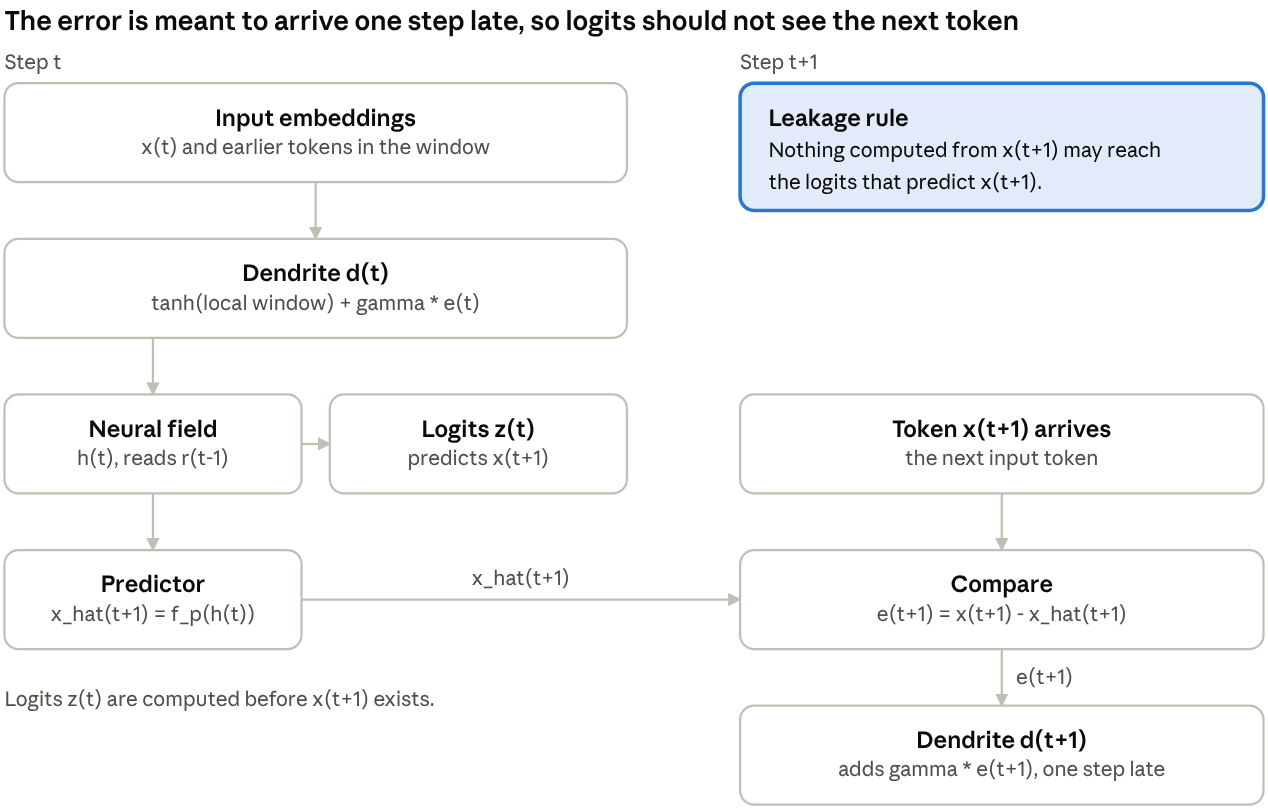
Block	Status	Main problem	Fix to try	First test
Slow memory and replay	Unproven; removing it did not change recall	Its read uses W′_q, the memory keys use W_k, so the spaces need not align; averaging unrelated sequences gives noise; two update rules act on W_s	Use W_q for the read; keep W_s out of weight decay; pick one update rule	Shared-facts task; continual learning, train A then B and measure A
Metacognition	Rules run; no evidence they help	Surprise is not difficulty; k is near constant on random data; round() has no gradient; the bias delta double counts surprise	Halt the field on convergence; fix one replay rule	Histogram of k; multi-hop task at equal FLOPs
Readout and logits head	Recall path works; order fixed from v1	Three terms of different scale in one sum; fixed mixing; memory can reach the logits through the field instead of the readout; vocabulary budget	Learned scalar per term; log each term′s norm; tie embeddings	Zero each term in turn; tied against untied embeddings

Every block that does not help on its own task is reported as "no evidence yet", not as useless. Metacognition is a rule-based controller and is described that way in this report.

Component diagrams

Four blocks are drawn in detail below: the dendrite with the predictor, the fast memory, the router, and the slow memory with replay.

Dendrite and predictor.

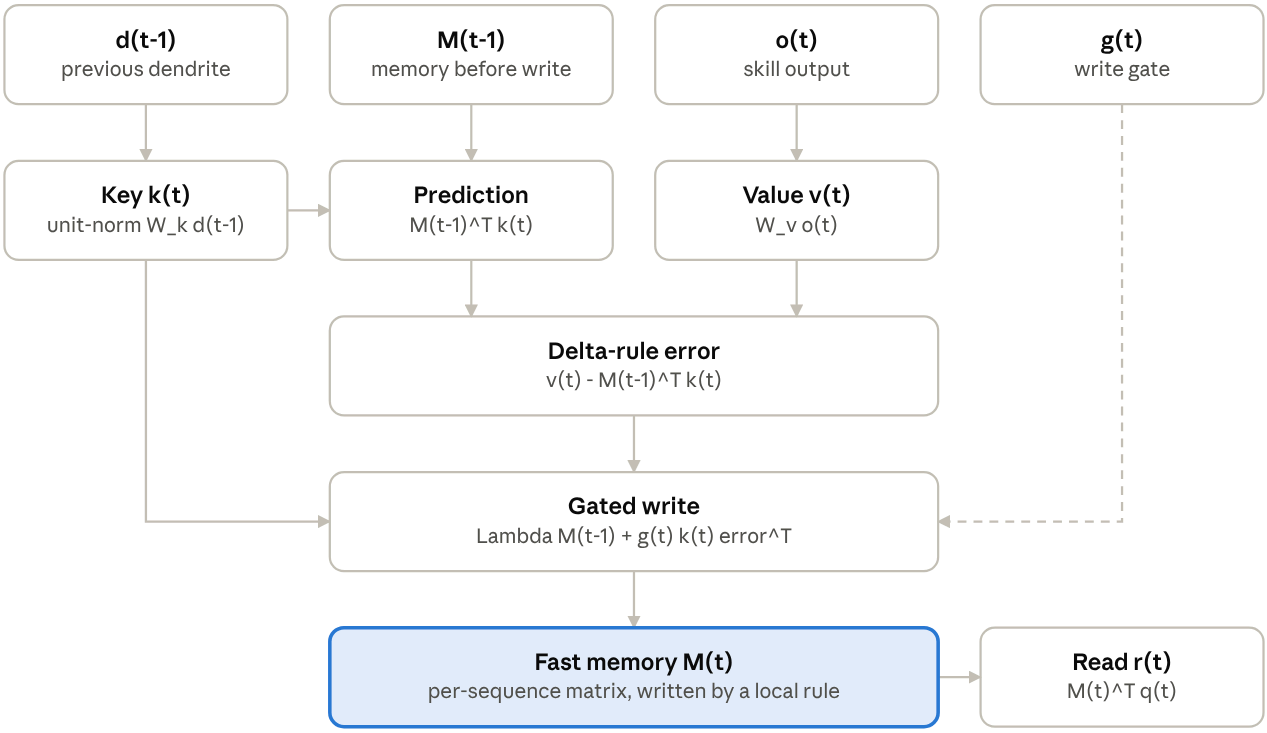


dendrite and predictor timing · two steps

The predictor forecasts x(t+1) at step t; the error is formed only when x(t+1) arrives and enters the dendrite at step t+1.

Fast memory.

A gated delta rule writes one key-value pair per token

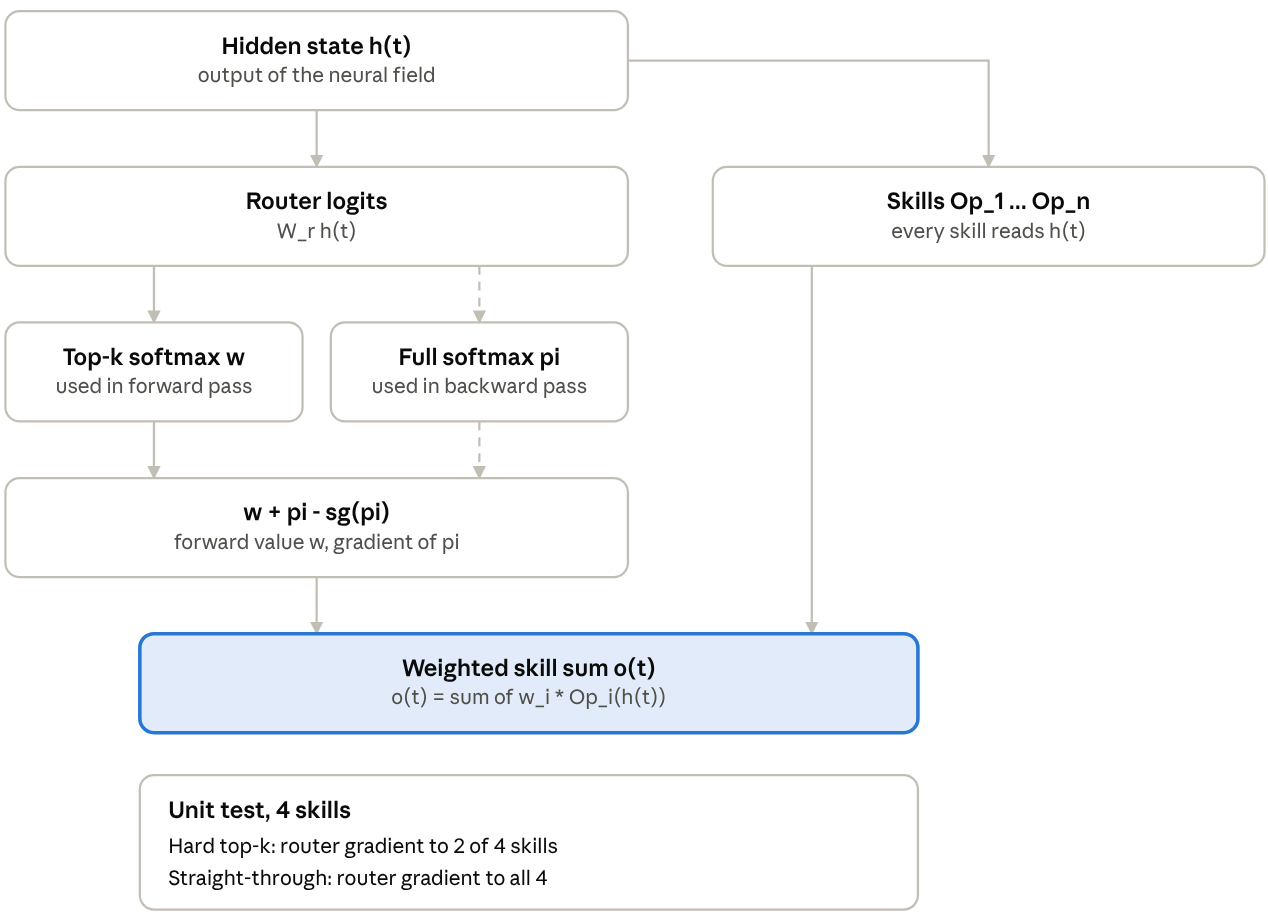


q(t) is the unit-norm W_q d(t). r(t) goes to the readout now and to the field at step t+1.

fast memory write and read · one token

The write stores the difference between the new value and what the memory already predicts for that key, scaled by the gate. The dashed arrow is the control signal from the neuromodulator.

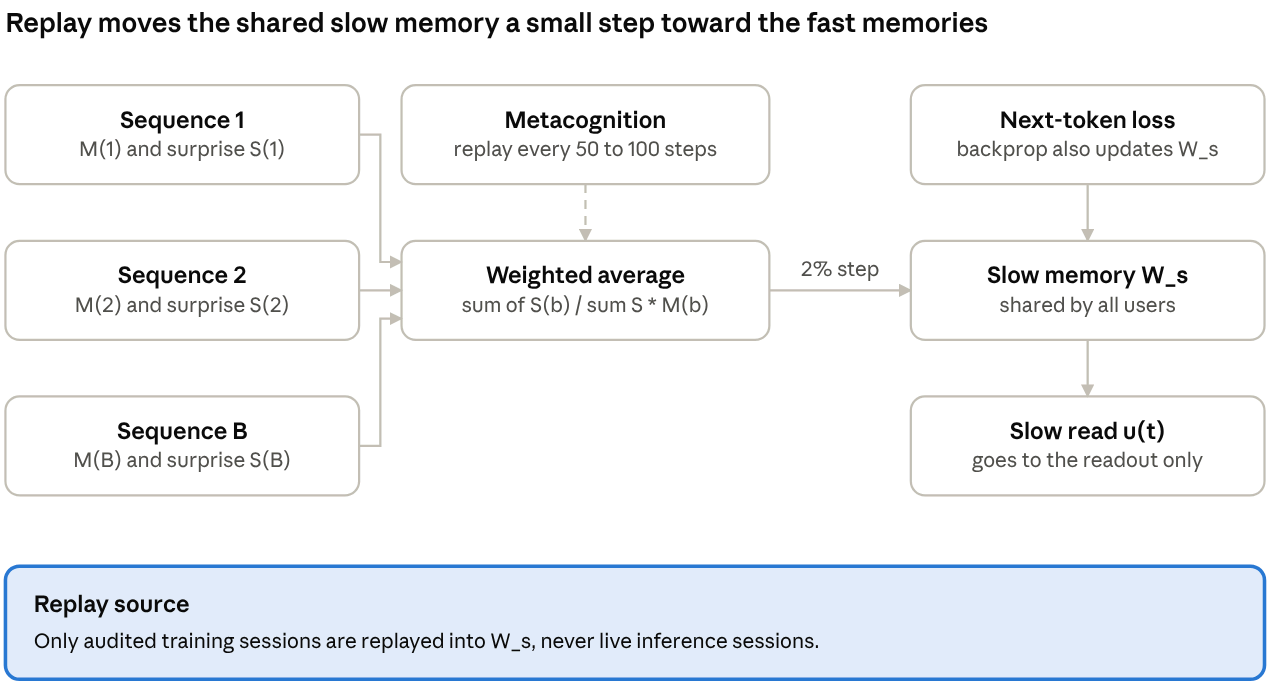
Router and skills.



router and skills · straight-through weights

The forward pass uses the top-k weights w, so unequal weights survive. The dashed arrows carry only the backward gradient of the full softmax. If every skill is still computed, as the audit suspects, the routing saves no compute (Section 5).

Slow memory and replay.



slow memory and replay · batch of sequences

Every 50 to 100 steps the slow weights move 2% toward the surprise-weighted average of the fast memories in the batch. Backprop on the next-token loss also updates the same weights.

6. Parameter efficiency

In the first 10M implementation the embedding and logits head held 72% of the parameters, leaving about 2.8M for the architecture itself. A 10,000-token vocabulary at width 368 costs 3.68M in each of the two matrices.

Model	Total params	Largest block	Share
First implementation (vocab 10,000)	10.16M	Embedding + head	72%
Reference AESC (char vocab 65)	9.92M	Skills (8 operators)	85%

Two fixes apply. Tie the input and output embeddings, which saves about 3.7M at that size, or use a small vocabulary (characters, or 2,000 to 4,000 subword tokens). v1 said subword tokens balance capacity; the opposite holds, because a larger vocabulary inflates both matrices.

In the reference model the new blocks are small: the field has 2.6K parameters, the fast memory 0.33M and the slow memory 0.07M. Skills dominate, so a control with the same skills and none of the new blocks is needed before crediting the architecture for any gain.

Compute-matched protocol. Report parameters, FLOPs per token, training tokens and wall-clock time for every model. The k-step field multiplies the field's FLOPs by k, so compare at equal FLOPs as well as at equal parameters. Tied embeddings also feed the predictor target, so the embedding norm is logged to catch collapse.

7. Safety and monitoring

AESC exposes internal signals that a monitor can log at every token. These signals are tripwires, not proof that the model's behavior is safe. This section is a design and has not been implemented or evaluated.

Signals logged per token.

- Surprise S(t) and its running average
- Write gate g(t), write norm, and the norm of the fast memory
- Router choices and router entropy
- Field steps k(t)
- Consolidation events into slow memory

Risks from test-time plasticity. The fast memory changes while the model is used. Content written in one session can steer later outputs (memory poisoning), the memory can drift without bound, and state shared across users can leak information.

Controls.

1. Fresh state for every session, with no carry-over between users.
2. Caps on the write gate, the write norm and the memory norm.
3. An audit log of writes: token index, gate, key norm and value norm.
4. An anomaly detector that flags signals far from a baseline distribution, for example a z-score on S(t), g(t) and router entropy.
5. A freeze switch that sets g = 0, so the memory only decays (Proposition 4), with an optional reset.
6. Replay into the shared slow weights only from audited training sessions, never from live inference sessions, so a poisoned write cannot become permanent.

Evaluation plan. Inject adversarial text that tries to plant false associations. Measure the detection rate, the false-alarm rate, and whether a reset removes the effect.

Scope. A monitor on these signals can show that something changed, but not why, and it is not a faithful trace of the model's reasoning. Safety claims need red-team evaluation, which this report does not contain.

8. Experiments and evaluation plan

The reference implementation solves associative recall with fast memory on, the first implementation did not, and no multi-seed or multi-task comparison exists yet. All runs below are single-seed, small-configuration CPU runs. The recall task has 4 key-value pairs, 8 keys and 8 values, with the answer at the last position (chance 0.125).

#	Experiment	Model	Result
1	Recall, 800 steps	Reference AESC (d = 64)	Accuracy 1.000
2	Recall, fast memory removed, 300-step ablation	Reference AESC	Accuracy about 0.40
3	Recall, stop-gradient write, 800 steps	Reference AESC	Accuracy 0.75; gate and key-map gradient exactly 0
4	Recall, fast updates on or off, 500 steps	First implementation (d = 64, 4 skills)	Accuracy 0.37 to 0.39 either way
5	Gradient audit, one backward pass	First implementation, 10.16M	Zero gradient in neuromodulator, metacognition, forgetting controller and field modulation input; router zero from the task loss
6	Gradient audit, one backward pass	Reference AESC, 9.92M	Nonzero gradient in all 11 blocks
7	Smoke run, char-level text, 200 steps, batch 8 x 32	AESC 9.92M, GRU 10.0M, Transformer about 10M	3.18, 2.91, 3.36 validation bits per character

Run 7 is too short to rank the models. Run 1 is consistent with the memory mechanism working on this task, but it is one seed on a toy task. Removing the slow memory, neuromodulator, metacognition, router or predictor did not change recall in the ablation, because this task does not need them, so there is no evidence yet that those blocks help.

Protocol for C2.

1. Tasks, one per block: recall with distractors and 8 to 64 pairs (fast memory); an overwrite task (decay); a noisy stream with few important tokens (neuromodulator); train task A then B and measure the drop on A (slow memory and replay); multi-hop lookup (field and metacognition); a task needing different operators (router); structured streams such as ABCABC against random tokens (predictor).
2. Models: the full AESC, a control with the same skills and none of the new blocks, one variant per removed block, a 10M transformer, and a 10M Mamba or GRU.
3. Same data, same training tokens, three or more seeds, mean and range reported, FLOPs per token listed.
4. A block counts as helping only if the full model beats the variant without it, outside the seed range, on the task built for that block. Otherwise the report says "no evidence yet".

Open items. Harder tasks that exercise the slow memory, neuromodulator and metacognition; the multi-seed runs above; GPU timing (the 10M model ran at about 0.96 s per step at batch 8 x 32 on CPU, and GPU speed is unmeasured); and the safety evaluation in Section 7.

9. Failure modes and fixes

Eleven failure modes are known. Rows 1 to 8 come from the audit of the first implementation, and rows 9 to 11 from review of v1 and follow-up tests.

#	Failure mode	Evidence	Fix
1	Fast weights averaged over the batch	One matrix shared by all sequences (code audit)	Per-sequence memory of shape batch x d_k x d_v
2	Writes frozen at inference	Update guarded by the training flag	Gated writes allowed at inference, scoped to the session and audited

#	Failure mode	Evidence	Fix
3	Neuromodulator off the graph	Zero gradient (audit)	Gate multiplies the differentiable write
4	Dendrite as autoencoder	It predicted features of its own input	Predict the next input; error arrives one step late
5	Router weights ignored	Selected skills averaged equally; zero router gradient from the task loss	Weighted skills with straight-through gradient
6	Field without memory read	Modulation input unused, zero gradient	Memory read fed into the field every step
7	Forgetting controller unused	Never called; zero gradient; fixed decay 0.95	Learnable per-key decay inside the write
8	Vocabulary swallows the budget	72% of 10.16M parameters	Tied embeddings or smaller vocabulary
9	Stop-gradient write blocks learning	Recall 0.75 vs 1.000; gate gradient 0	Differentiable write in a truncated window
10	Possible label leakage	v1 equation e(t) = x(t+1) - pred (design review, not measured)	One-step delay; unit test in Appendix B
11	Figure 1 errors	Softmax before logits head, garbled label, unexplained N-times stack	Redrawn (Section 3)

10. Limitations and non-claims

No result here shows that a small AESC matches a larger model on any task. The evidence comes from small configurations, single seeds and one synthetic task.

- Only the fast memory is supported by an experiment. The slow memory, neuromodulator, metacognition, router and predictor are unproven (claim C2 is untested).
- The component audit in Section 5 is design-level: it was done from the report and its equations, not from the implementation code.

- The model processes tokens one at a time, so training is slower than a transformer of equal size; GPU throughput is unmeasured.
- Metacognition is a set of simple rules, not a learned policy.
- The safety layer is a design, not a tested system.
- The author does not claim biological fidelity, convergence, unique skills or superiority to transformers.

11. Future work

The main proposal is a per-user context memory that keeps a long chat from being lost, without writing one user's chat into the shared slow weights. Everything in this section is a proposal and has not been tested.

Per-user context memory C_u. The fast memory M loses old context through decay, and the slow memory W_s is shared by all users. A third memory C_u, private to one user, sits between them:

```
C_u \leftarrow (1-\eta)\, C_u + \eta \sum_b \frac{S^{(b)}}{\sum_{b'} S^{(b')}}\, M^{(b)}
```

- During a chat, M writes as before. At the end of each turn, the important part of M is consolidated into C_u, with a norm cap and an audit log.
- A new turn or session starts from M_0 = C_u, or adds a read term P_u C_u^T q to the readout.
- C_u is isolated by user ID, and the user can reset it.
- W_s is never written from a live chat. This keeps the rule of Section 7 and adds one exception: live consolidation is allowed only into the user's private C_u.
- Test: state facts in session 1 and ask about them in session 2, with C_u on and off, over three or more seeds.

Other items from the audit.

- A slow-memory term in the field update, ablated against the current design.
- Convergence-based halting for the field steps in place of the rounded surprise rule.
- A content-dependent write gate.
- A learned controller to replace the Metacognition rules.

Appendix A. Forward equations

Symbols: x_t is the input embedding, h the field state, M_t the fast memory, and sg the stop-gradient operator. The equations match the reference implementation.

Dendrite and prediction.

```
d_t = \tanh\Big(\sum_{j=0}^{w-1} c_j \odot x_{t-j} + b\Big) + \gamma \odot
e_t
```

```
\hat{x}_{t+1} = f_p(h_t), \qquad e_{t+1} = x_{t+1} - \hat{x}_{t+1}, \qquad
S_{t+1} = \mathrm{sg}\big(\mathrm{mean}\,|e_{t+1}|\big)
```

Field (repeated k_t times). R is a shared local convolution over neighboring neurons.

```
h \leftarrow (1-\zeta)\odot h + \zeta \odot \mathrm{LN}\Big(\alpha \odot h +
\beta \odot \big(d_t + R(\tanh h) + W_m r_{t-1}\big)\Big)
```

Control. The running average of surprise is S_bar, and the relative surprise is S_tilde_t = S_t / S_bar. The gate bias is delta_t = +0.5 if S_tilde_t > 1.5, -0.5 if S_tilde_t < 0.5, and 0 otherwise.

```
g_t = \sigma\big(a_s(\tilde S_t - 1) + a_c\,\overline{|h_t|} + b +
\delta_t\big)
```

```
k_t = 1 + \mathrm{round}\Big((k_{max}-1)\,\mathrm{clip}\big(\tfrac{\tilde S_t
- 0.5}{1.5},\,0,\,1\big)\Big)
```

Router and skills.

```
\pi = \mathrm{softmax}(W_r h_t), \quad w = \mathrm{softmax}(\text{top-}k\
\text{logits}), \quad w^{STE} = w + \pi - \mathrm{sg}(\pi), \quad o_t =
\sum_i w^{STE}_i\,\mathrm{Op}_i(h_t)
```

Fast memory. Lambda is a diagonal decay over key dimensions.

```
k_t = \frac{W_k d_{t-1}}{\lVert W_k d_{t-1} \rVert}, \qquad v_t = W_v o_t,
\qquad M_t = \Lambda M_{t-1} + g_t\, k_t \big(v_t - M_{t-1}^{\top}
k_t\big)^{\top}
```

Read and output.

```
q_t = \frac{W_q d_t}{\lVert W_q d_t \rVert}, \quad r_t = M_t^{\top} q_t,
\quad u_t = (W'_q d_t)^{\top} W_s, \quad z_t = W_o\,\mathrm{LN}\big(o_t + P_f
r_t + P_s u_t\big)
```

Replay into slow memory.

```
W_s \leftarrow (1-\eta)\,W_s + \eta \sum_b \frac{S^{(b)}}{\sum_{b'}
S^{(b')}}\, M_t^{(b)}
```

Appendix B. Design checklist

1. Per-sequence fast state of shape batch x d_k x d_v.
2. The gate multiplies the write; writes are allowed at inference and are audited.
3. Router weights are used in the forward pass, and every skill receives a straight-through gradient.
4. The dendrite predicts the next input, and the error is computed one step late.
5. The field reads the memory at every step.
6. The neuromodulator and gate receive gradient: after one backward pass on the next-token loss alone, their gradient norm is above zero.
7. The recall test passes (accuracy of at least 0.95 on 4-pair recall) before any other claim is made.
8. Leakage test: change x(t+1) and check that the logits at step t do not change.
9. The stop-gradient ablation is reported next to the differentiable write.
10. A per-block parameter report is included, with embedding and head at 25% of the total or less (a target), or tied.
11. Baselines are compared at equal FLOPs as well as equal parameters.
12. The safety log fields are present: S(t), g(t), router ids, k(t), the memory norm and the write audit.
13. Each row of the audit table in Section 5 has a measured result, or is marked "no evidence yet".
14. The embedding norm is logged per epoch, so a collapse caused by the error loss is visible.

Appendix C. Propositions

These follow from the chain rule and the construction. They are not convergence results.

Proposition 1 (reachability). If a module's output enters the logits or the error through a differentiable map with a nonzero Jacobian, the next-token loss gives that module a gradient that can be nonzero. Evidence: Run 6 shows nonzero gradient in all 11 blocks of the reference model.

Proposition 2 (straight-through routing). Let w be the softmax over the top-k logits and pi the full softmax. Then w + pi - sg(pi) equals w in the forward pass, and its gradient is the gradient of pi. Every skill therefore receives a router gradient. This replaces the uniform 1/k forward weights of v1, which cannot express unequal skill weights.

Proposition 3 (revised: stop-gradient writes block learning). If the write M_t = Lambda M_{t-1} + g k (v - M^T k)^T is computed under stop-gradient, then M_t is constant with respect to every parameter, and W_k, W_v and the gate parameters receive no gradient through M. Observation: in Run 3 the gradient was exactly 0 and recall fell from 1.000 to 0.75. v1 claimed the stop-gradient design was justified; that claim is withdrawn.

Proposition 4 (zero gate). If g_t = 0 then M_t = Lambda M_{t-1}, so the memory only decays.

Proposition 5 (inference plasticity). With fixed parameters and g_t > 0, M changes across tokens during inference. This is why Section 7 requires per-session state and a write audit.

Proposition 6 (error supervision). The term lambda_err times the squared norm of e(t+1) sends gradient to the predictor and, through the state, to the field and dendrite. Surprise S is detached, so it carries no gradient.

Proposition 7 (no leakage). If the logits z_t depend on the inputs only through x(1..t), and errors are computed with a one-step delay, then the derivative of z_t with respect to x(t+1) is zero. The reference implementation processes one token per step, so causality should hold by construction; the Appendix B test has not been run yet.